

**DATE:****Aim:**

To understand and implement the fundamental React Hooks (useState, useEffect, useContext, useCallback, useMemo, useRef, and Custom Hooks) by developing simple real-world applications for state management, side effects, shared data, performance optimization, DOM manipulation, and reusable logic.

**9.a useState****Problem Statement 1:**

You are developing a quiz app where the user's selected answer must be stored for each question. How would you use useState to track the chosen options?

**Program:****Quiz.jsx**

```
import { useState } from "react";

function Quiz() {

  const [selectedAnswer, setSelectedAnswer] = useState("");

  const options = ["Java", "Python", "JavaScript", "C++"];

  return (

    <div>

      <h2>Quiz Application</h2>

      <h3>Which language is used for React?</h3>

      {options.map((option) => (

        <div key={option}>

          <input

            type="radio"

            value={option}

            name="quiz"

            checked={selectedAnswer === option}

            onChange={(e) => setSelectedAnswer(e.target.value)}

          />

          <label>{option}</label>

        </div>

      )

    )}
```

```

    </div>

  )}

  <h3>Selected Answer: {selectedAnswer}</h3>

</div>

);

}

export default Quiz;

App.jsx

import Quiz from "../components/Quiz";

function App() {
  return <Quiz />;
}

export default App;

```

**Output:**

---

## Quiz Application

**Which language is used for React?**

- ☐ Java
- ☐ Python
- ☒ JavaScript
- ☐ C++

**Selected Answer: JavaScript**

### Problem Statement 2:

In a shopping app, you want to maintain the list of items added to the cart. How can useState help manage this dynamic array?

**Program:**

**ShoppingCart.jsx**

```
import { useState } from "react";
```

```

function ShoppingCart() {
  const [cart, setCart] = useState([]);
  const addItem = (item) => {
    setCart([...cart, item]);
  };
  return (
    <div style={{ padding: "20px" }}>
      <h2>Shopping Cart</h2>
      <button onClick={() => addItem("Laptop")}>
        Add Laptop
      </button>
      <button onClick={() => addItem("Mobile")}>
        Add Mobile
      </button>
      <button onClick={() => addItem("Headphones")}>
        Add Headphones
      </button>
      <h3>Items in Cart</h3>
      <ul>
        {cart.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
      <h3>Total Items: {cart.length}</h3>
    </div>
  );
}

export default ShoppingCart;

```

**App.jsx**

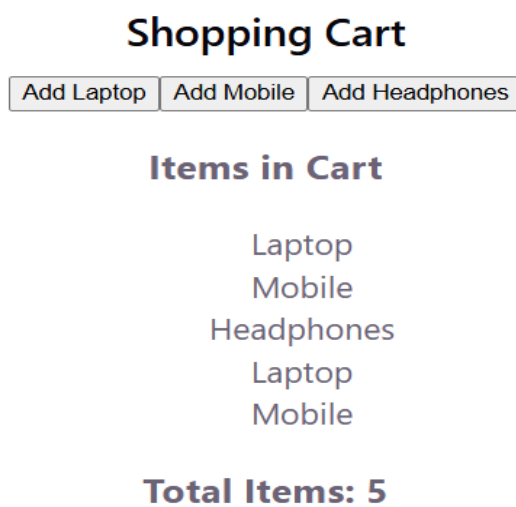
```

import ShoppingCart from "../components/ShoppingCart";

```

```
function App() {
  return (
    <div>
      <ShoppingCart />
    </div>
  );
}
export default App;
```

### Output:



### Problem Statement 3:

You are building a toggle button for showing/hiding a modal popup. How would you use useState to track the modal's visibility state?

### Program:

#### Model.jsx

```
import { useState } from "react";

function Modal() {
  const [showModal, setShowModal] = useState(false);

  return (
    <div
      style={{
        textAlign: "center",
```

```

      marginTop: "40px",
    }}
  >
  <h2>Modal Popup Example</h2>
  <button onClick={() => setShowModal(!showModal)}>
    {showModal ? "Hide Modal" : "Show Modal"}
  </button>
  {showModal && (
    <div
      style={{
        width: "320px",
        margin: "25px auto",
        padding: "20px",
        border: "1px solid black",
        borderRadius: "8px",
        backgroundColor: "#f2f2f2",
        boxShadow: "0 2px 8px rgba(0,0,0,0.2)",
      }}
    >
      <h2>Welcome!</h2>
      <p>This is a modal popup.</p>
      <button onClick={() => setShowModal(false)}>
        Close
      </button>
    </div>
  )}
</div>
);
}
export default Modal;

```

### App.jsx

```
import Modal from "../components/Modal";

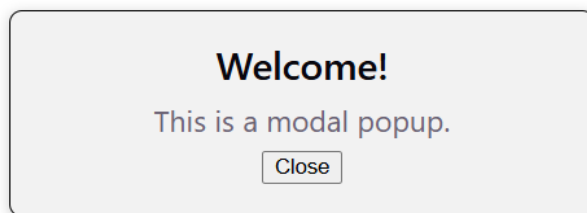
function App() {
  return (
    <div>
      <Modal />
    </div>
  );
}

export default App;
```

### Output:

#### Modal Popup Example

Hide Modal



#### Modal Popup Example

Show Modal

### 9.b useEffect

#### Problem Statement 1:

In a weather app, you need to fetch live weather data from an API whenever the page loads. How can you use useEffect to achieve this?

#### Program:

#### Weather.jsx

```
import { useState, useEffect } from "react";
```

```

function Weather() {
  const [weather, setWeather] = useState(null);
  useEffect(() => {
    fetch(
      "https://api.open-
meteo.com/v1/forecast?latitude=13.08&longitude=80.27&current_weather=true"
    )
      .then((response) => response.json())
      .then((data) => setWeather(data.current_weather))
      .catch((error) => console.log(error));
  }, []);
  return (
    <div style={{ textAlign: "center", marginTop: "30px" }}>
      <h2>Weather Application</h2>
      {weather ? (
        <div>
          <h3>Current Weather</h3>
          <p>Temperature: {weather.temperature} °C</p>
          <p>Wind Speed: {weather.windspeed} km/h</p>
        </div>
      ) : (
        <h3>Loading Weather...</h3>
      )}
    </div>
  );
}

```

```
export default Weather;
```

### **App.jsx**

```
import Weather from "../components/Weather";
```

```
function App() {
```

```
  return (
```

```
    <div>
```

```

    <Weather />
  </div>
);
}
export default App;

```

**Output:**

## Weather Application

### Current Weather

Temperature: 32.6 °C  
Wind Speed: 9.6 km/h

### Problem Statement 2:

You are tracking a user's scroll position to show a "Back to Top" button. How would you use `useEffect` to add and clean up the scroll event listener?

### Program:

#### ScrollTracker.jsx

```

import { useState, useEffect } from "react";

function ScrollTracker() {
  const [showButton, setShowButton] = useState(false);
  useEffect(() => {
    const handleScroll = () => {
      if (window.scrollY > 50) {
        setShowButton(true);
      } else {
        setShowButton(false);
      }
    };
  });
}

```

```

window.addEventListener("scroll", handleScroll);
return () => {
  window.removeEventListener("scroll", handleScroll);
};
}, []);
const scrollToTop = () => {
  window.scrollTo({
    top: 0,
    behavior: "smooth",
  });
};
return (
  <div style={{ padding: "20px" }}>
    <h2 style={{ textAlign: "center" }}>Scroll Tracker</h2>
    <p style={{ textAlign: "center" }}>
      Scroll down to view the "Back to Top" button.
    </p>
    {Array.from({ length: 40 }, (_, index) => (
      <p key={index}>Item {index + 1}</p>
    ))}
    {showButton && (
      <button
        onClick={scrollToTop}
        style={{
          position: "fixed",
          bottom: "20px",
          left: "50%",
          transform: "translateX(-50%)",
          padding: "10px 20px",
          backgroundColor: "blue",
          color: "white",

```

```

    border: "none",
    borderRadius: "5px",
    cursor: "pointer",
    fontSize: "16px",
  }}
>
  Back to Top
</button>
)}
</div>
);
}
export default ScrollTracker;

```

### App.jsx

```

import ScrollTracker from "../components/ScrollTracker";

function App() {
  return (
    <div>
      <ScrollTracker />
    </div>
  );
}

export default App;

```

### Output:

#### Scroll Tracker

Scroll down to view the "Back to Top" button.

Item 1  
 Item 2  
 Item 3  
 Item 4  
 Item 5  
 Item 6  
 Item 7  
 Item 8  
 Item 9  
 Item 10  
 Item 11

Item 12  
Item 13  
Item 14  
Item 15  
Item 16  
Item 17  
Item 18  
Item 19  
Item 20  
Item 21  
Item 22  
Item 23  
Item 24  
Item 25  
Item 26  
Item 27  
Item 28  
Item 29  
Item 30  
Item 31  
Item 32  
Item 33  
Item 34  
Item 35

[Back to Top](#)

Item 37

### Problem Statement 3:

In a product details page, you need to fetch product info from the backend whenever the productId changes. How would you use useEffect to refetch data based on this dependency?

### Program:

#### Product.jsx

```
import React, { useEffect, useState } from "react";

function Product({ productId }) {

  const [product, setProduct] = useState(null);

  useEffect(() => {

    fetch(`https://fakestoreapi.com/products/${productId}`)

    .then((response) => response.json())

    .then((data) => {

      setProduct(data);

    })

    .catch((error) => {
```

```

        console.log(error);
    });
}, [productId]);
return (
    <div>
        <h2>Product Details</h2>
        {
            product ? (
                <div>
                    <img
                        src={product.image}
                        width="150"
                        alt="product"
                    />
                    <h3>{product.title}</h3>
                    <p>Price: ${product.price}</p>
                    <p>Category: {product.category}</p>
                </div>
            )
            :
            (
                <h3>Loading...</h3>
            )
        }
    </div>
);
}
export default Product;

```

### **App.jsx**

```

import React, { useState } from "react";
import Product from "../components/Product";

```

```

function App() {
  const [productId, setProductId] = useState(1);
  return (
    <div>
      <h1>Product Store</h1>
      <button onClick={() => setProductId(productId + 1)}>
        Next Product
      </button>
      <p>
        Current Product ID: {productId}
      </p>
      <Product productId={productId}/>
    </div>
  );
}
export default App;

```

**Output:**

# Product Store

Next Product

Current Product ID: 7

## Product Details



### White Gold Plated Princess

Price: \$9.99

Category: jewellery

# Product Store

Next Product

Current Product ID: 13

## Product Details



**Acer SB220Q bi 21.5 inches Full HD (1920 x 1080) IPS Ultra-Thin**

Price: \$599

Category: electronics

## 9.c useContext

### Problem Statement 1:

In a dashboard app, you need to share the logged-in user's profile information across multiple nested components. How can useContext help simplify this instead of prop drilling?

### Program:

#### Dashboard.jsx

```
import React, { useContext } from "react";
import { UserContext } from "../App";

function Dashboard() {
  const user = useContext(UserContext);
  return (
    <div>
      <h1>Dashboard</h1>
      <h2>User Profile</h2>
      <p>Name: {user.name}</p>
      <p>Email: {user.email}</p>
      <p>Role: {user.role}</p>
    </div>
  );
}

export default Dashboard;
```

### App.jsx

```
import React, { createContext } from "react";
import Dashboard from "../components/Dashboard";
export const UserContext = createContext();
function App() {
  const user = {
    name: "Alice",
    email: "alice@gmail.com",
    role: "Student"
  };
  return (
    <UserContext.Provider value={user}>
      <Dashboard />
    </UserContext.Provider>
  );
}
export default App;
```

### Output:

# Dashboard

## User Profile

Name: Alice  
Email: [alice@gmail.com](mailto:alice@gmail.com)  
Role: Student

### Problem Statement 2:

A shopping cart has to display the total number of items in the header, sidebar, and footer simultaneously. How would you use useContext to manage this shared state?

### Program:

### ShoppingCart.jsx

```
import React, { useContext } from "react";
import { CartContext } from "../App";
```

```

function ShoppingCart() {
  const totalItems = useContext(CartContext);
  return (
    <div style={{ textAlign: "center", marginTop: "40px" }}>
      <h1>Shopping Cart</h1>
      <h2>Header</h2>
      <p>Total Items: {totalItems}</p>
      <h2>Sidebar</h2>
      <p>Total Items: {totalItems}</p>
      <h2>Footer</h2>
      <p>Total Items: {totalItems}</p>
    </div>
  );
}
export default ShoppingCart;

```

### **App.jsx**

```

import React, { createContext } from "react";
import ShoppingCart from "../components/ShoppingCart";
export const CartContext = createContext();

function App() {
  const totalItems = 5;
  return (
    <CartContext.Provider value={totalItems}>
      <ShoppingCart />
    </CartContext.Provider>
  );
}
export default App;

```

## Output:

# Shopping Cart

Header

Total Items: 5

Sidebar

Total Items: 5

Footer

Total Items: 5

## 9.d useCallback

### Problem Statement 1:

You have a list of users, and each user has a “Follow” button. The click handler is defined inside the parent component, causing performance issues. How would you optimize it using useCallback?

### Program:

#### Users.jsx

```
import React from "react";

function Users({ users, onFollow }) {
  return (
    <div>
      {users.map((user, index) => (
        <div key={index}>
          <h3>{user}</h3>
          <button onClick={() => onFollow(user)}>
            Follow
          </button>
        </div>
      ))}
    </div>
  );
}

export default React.memo(Users);
```

## App.jsx

```
import React, { useState, useCallback } from "react";
import Users from "../components/Users";

function App() {
  const [count, setCount] = useState(0);
  const users = ["Alice", "Bob", "Charlie"];
  const handleFollow = useCallback((name) => {
    alert("You followed " + name);
  }, []);
  return (
    <div style={{ textAlign: "center", marginTop: "30px" }}>
      <h1>User List</h1>
      <button onClick={() => setCount(count + 1)}>
        Counter : {count}
      </button>
      <Users users={users} onFollow={handleFollow} />
    </div>
  );
}

export default App;
```

## Output:

# User List

Counter : 0

**Alice**

Follow

**Bob**

Follow

**Charlie**

Follow

localhost:5173 says

You followed Alice

OK

Counter : 1

**Alice**

Follow

**Bob**

Follow

**Charlie**

Follow

## Problem Statement 2:

You are working on a form with multiple fields. Each input has an onChange handler passed from the parent. How can useCallback help reduce unnecessary re-renders of these input components?

### Program:

#### Form.jsx

```
import React from "react";

function Form({ formData, handleChange }) {
  return (
    <div>
      <input
        type="text"
        name="name"
        placeholder="Enter Name"
        value={formData.name}
        onChange={handleChange}
      />
      <br /><br />
      <input
        type="email"
        name="email"
        placeholder="Enter Email"
        value={formData.email}
        onChange={handleChange}
      />
      <br /><br />
      <h3>Name: {formData.name}</h3>
      <h3>Email: {formData.email}</h3>
    </div>
  );
}

export default React.memo(Form);
```

## App.jsx

```
import React, { useState, useCallback } from "react";
import Form from "../components/Form";

function App() {
  const [formData, setFormData] = useState({
    name: "",
    email: ""
  });
  const handleChange = useCallback((e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({
      ...prev,
      [name]: value
    }));
  }, []);
  return (
    <div style={{ textAlign: "center", marginTop: "30px" }}>
      <h1>User Form</h1>
      <Form formData={formData} handleChange={handleChange} />
    </div>
  );
}

export default App;
```

## Output:

### User Form

**Name: Alice**

**Email: alice@gmail.com**

### User Form

**Name: Charlie**

**Email: charlie@gmail.com**

### Problem Statement 3:

In a to-do app, the child component renders a list of tasks and receives a deleteTask function from the parent. How can you ensure this function reference stays stable across renders using useCallback?

#### Program:

##### Todo.jsx

```
import React from "react";

function Todo({ tasks, deleteTask }) {
  return (
    <div>
      {tasks.map((task, index) => (
        <div key={index}>
          <p>{task}</p>
          <button onClick={() => deleteTask(index)}>
            Delete
          </button>
          <br /><br />
        </div>
      ))}
    </div>
  );
}
```

```
export default React.memo(Todo);
```

##### App.jsx

```
import React, { useState, useCallback } from "react";
import Todo from "../components/Todo";

function App() {
  const [tasks, setTasks] = useState([
    "Study React",
    "Complete Lab",
    "Submit Assignment"
  ]);
}
```

```

const deleteTask = useCallback((index) => {
  setTasks((prevTasks) =>
    prevTasks.filter((_, i) => i !== index)
  );
}, []);
return (
  <div style={{ textAlign: "center", marginTop: "30px" }}>
    <h1>To-Do List</h1>

    <Todo tasks={tasks} deleteTask={deleteTask} />
  </div>
);
}
export default App;

```

### Output:

# To-Do List

Study React

Delete

Complete Lab

Delete

Submit Assignment

Delete

## 9.e useMemo

### Problem Statement 1:

In a financial dashboard, calculating the total revenue requires iterating over thousands of records. How can you prevent recalculating it on every render using useMemo?

### Program:

#### Revenue.jsx

```
import React, { useMemo } from "react";
```

```

function Revenue({ records }) {
  const totalRevenue = useMemo(() => {
    console.log("Calculating Revenue...");
    return records.reduce((total, record) => total + record.amount, 0);
  }, [records]);
  return (
    <div>
      <h2>Total Revenue</h2>
      <h3>₹ {totalRevenue}</h3>
    </div>
  );
}
export default Revenue;

```

### App.jsx

```

import React, { useState } from "react";
import Revenue from "../components/Revenue";
function App() {
  const [count, setCount] = useState(0);
  const records = [
    { id: 1, amount: 5000 },
    { id: 2, amount: 7000 },
    { id: 3, amount: 3000 },
    { id: 4, amount: 9000 },
    { id: 5, amount: 6000 }
  ];
  return (
    <div style={{ textAlign: "center", marginTop: "30px" }}>
      <h1>Financial Dashboard</h1>
      <button onClick={() => setCount(count + 1)}>
        Counter : {count}
      </button>
    </div>
  );
}

```

```

    <Revenue records={records} />
  </div>

);
}

```

export default App;

**Output:**

## Financial Dashboard

Counter : 0  
Total Revenue  
₹ 30000

## Financial Dashboard

Counter : 2  
Total Revenue  
₹ 30000

### Problem Statement 2:

You are building a search bar that filters a large dataset of products. The filtering logic is slow. How can you optimize this so filtering only happens when the search term changes?

### Program:

#### Search.jsx

```

import React, { useState, useMemo } from "react";

function Search({ products }) {
  const [searchTerm, setSearchTerm] = useState("");
  const filteredProducts = useMemo(() => {
    console.log("Filtering Products...");
    return products.filter((product) =>
      product.toLowerCase().includes(searchTerm.toLowerCase())
    );
  }, [searchTerm, products]);
}

```

```

    );
  }, [searchTerm, products]);
  return (
    <div>
      <input
        type="text"
        placeholder="Search Product"
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
      />
      <h2>Products</h2>
      {filteredProducts.map((product, index) => (
        <p key={index}>{product}</p>
      )))}
    </div>
  );
}
export default Search;

```

### **App.jsx**

```

import React from "react";
import Search from "../components/Search";
function App() {
  const products = [
    "Laptop",
    "Mobile",
    "Mouse",
    "Keyboard",
    "Monitor",
    "Headphones",
    "Charger",
    "Speaker"
  ]

```

```

];
return (
  <div style={{ textAlign: "center", marginTop: "30px" }}>
    <h1>Product Search</h1>
    <Search products={products} />
  </div>
);
}
export default App;

```

### Output:

## Product Search

**Products**

- Laptop
- Mobile
- Mouse
- Keyboard
- Monitor
- Headphones
- Charger
- Speaker

## Product Search

**Products**

- Keyboard

### 9.f useRef

#### Problem Statement 1:

Track Previous State : Create a counter (count) that increases on button click. Use useRef to store the previous value of count. Display both the current and previous values.

#### Program:

##### Counter.jsx

```

import React, { useState, useRef } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  const previousCount = useRef(0);
  const increaseCount = () => {
    previousCount.current = count;
    setCount(count + 1);
  }
}

```

```

};

return (
  <div style={{ textAlign: "center", marginTop: "40px" }}>
    <h1>Counter App</h1>
    <h2>Current Count: {count}</h2>
    <h2>Previous Count: {previousCount.current}</h2>
    <button onClick={increaseCount}>
      Increase
    </button>
  </div>
);
}

export default Counter;

```

### App.jsx

```

import React from "react";
import Counter from "../components/Counter";

function App() {
  return (
    <div>
      <Counter />
    </div>
  );
}

export default App;

```

### Output:

## Counter App

Current Count: 0

Previous Count: 0

Increase

## Counter App

Current Count: 2

Previous Count: 1

Increase

## Problem Statement 2:

Scroll to an Element: Create a long list of items. Use useRef to point to a specific item (e.g., the 50th item). Add a button that scrolls directly to that item.

### Program:

#### Scroll.jsx

```
import React, { useRef } from "react";

function Scroll() {
  const itemRef = useRef(null);

  const scrollToItem = () => {
    itemRef.current.scrollToView({
      behavior: "smooth"
    });
  };

  return (
    <div style={{ textAlign: "center", marginTop: "20px" }}>
      <h1>Scroll to an Item</h1>
      <button onClick={scrollToItem}>
        Go to Item 50
      </button>
      <div
        style={{
          height: "300px",
          overflowY: "scroll",
          border: "1px solid black",
          marginTop: "20px"
        }}
      >
        {[...Array(100)].map((_, index) => (
          <div
            key={index}
            ref={index === 49 ? itemRef : null}
            style={{
```

```

        padding: "15px",
        borderBottom: "1px solid gray"
      }}
    >
    Item {index + 1}
  </div>
))}
</div>
</div>
);
}
export default Scroll;

```

### App.jsx

```

import React from "react";
import Scroll from "../components/Scroll";
function App() {
  return (
    <div>
      <Scroll />
    </div>
  );
}
export default App;

```

### Output:

## Scroll to an Item

Go to Item 50

|        |
|--------|
| Item 1 |
| Item 2 |
| Item 3 |
| Item 4 |
| Item 5 |

## Scroll to an Item

Go to Item 50

|         |
|---------|
| Item 50 |
| Item 51 |
| Item 52 |
| Item 53 |
| Item 54 |

## 9.g Customized Hooks

### Problem Statement 1:

useDebounce Hook: Accept a value and delay. Return the debounced value (only update after no changes for given delay).

### Program:

#### useDebounce.jsx

```
import { useState, useEffect } from "react";

function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);
    return () => clearTimeout(timer);
  }, [value, delay]);
  return debouncedValue;
}

export default useDebounce;
```

#### Search.jsx

```
import React, { useState } from "react";
import useDebounce from "../hooks/useDebounce";

function Search() {
  const [search, setSearch] = useState("");
  const debouncedSearch = useDebounce(search, 1000);
  return (
    <div style={{ textAlign: "center", marginTop: "40px" }}>
      <h1>Search Products</h1>
      <input
        type="text"
        placeholder="Search..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />
    </div>
  );
}
```

```

    />

    <h3>Typed Value: {search}</h3>

    <h3>Debounced Value: {debouncedSearch}</h3>

  </div>

);

}

export default Search;

```

### App.jsx

```

import React from "react";

import Search from "../components/Search";

function App() {
  return (
    <div>
      <Search />
    </div>
  );
}

export default App;

```

### Output:

## Search Products

**Typed Value: Laptop**

**Debounced Value: Laptop**

## Search Products

**Typed Value: Keyboard**

**Debounced Value: Keyboard**

## Problem Statement 2:

useEventListener Hook: Create a hook that attaches/detaches an event listener to any target (window, document, ref.current).

### Program:

#### useEventListener.jsx

```
import { useEffect } from "react";

function useEventListener(eventType, handler, target = window) {
  useEffect(() => {
    target.addEventListener(eventType, handler);

    return () => {
      target.removeEventListener(eventType, handler);
    };
  }, [eventType, handler, target]);
}

export default useEventListener;
```

#### MouseTracker.jsx

```
import React, { useState } from "react";
import useEventListener from "../hooks/useEventListener";

function MouseTracker() {
  const [position, setPosition] = useState({ x: 0, y: 0 });
  const handleMouseMove = (event) => {
    setPosition({
      x: event.clientX,
      y: event.clientY
    });
  };

  useEventListener("mousemove", handleMouseMove);

  return (
    <div style={{ textAlign: "center", marginTop: "40px" }}>
      <h1>Mouse Position Tracker</h1>
      <h2>X: {position.x}</h2>
      <h2>Y: {position.y}</h2>
    </div>
  );
}
```

```

    <p>Move the mouse anywhere on the screen.</p>
  </div>

);
}
export default MouseTracker;

App.jsx
import React from "react";
import MouseTracker from "../components/MouseTracker";

function App() {
  return (
    <div>
      <MouseTracker />
    </div>
  );
}
export default App;

```

**Output:**

## Mouse Position Tracker

X: 953

Y: 566

Move the mouse anywhere on the screen.

---

## Mouse Position Tracker

X: 905

Y: 541

Move the mouse anywhere on the screen.

| <b>Problem Understanding &amp; Approach (10)</b> | <b>Implementation &amp; Execution (15)</b> | <b>Tool/ Technology Usage (10)</b> | <b>Code Quality &amp; Documentation (5)</b> | <b>Viva (5)</b> | <b>Time Management (5)</b> | <b>Total (50)</b> |
|--------------------------------------------------|--------------------------------------------|------------------------------------|---------------------------------------------|-----------------|----------------------------|-------------------|
|                                                  |                                            |                                    |                                             |                 |                            |                   |

### **Result:**

Thus, the fundamental React Hooks (useState, useEffect, useContext, useCallback, useMemo, useRef, and Custom Hooks) were successfully implemented and tested. The desired output was obtained for state management, side effects, shared data, performance optimization, DOM manipulation, and reusable logic.